

Clase 4: SOLID

Fer Cargnelutti



5 Principles



- ✓ Single Responsibility
- ✓ Open/Closed
- ✓ Liskov Substitution
- ✓ Interface Segregation
- ✓ Dependency Inversion

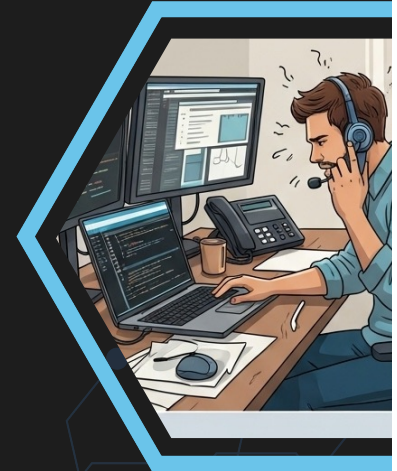
"Solid principles are about dependencies between classes and its relations. "

SOLID

Principle (SOLID)	Inver
S - Single Responsibility	Robert C
O - Open/Closed	Robert C

Single Responsibility

- The responsibility that the classes have to they users.
- Only one reason to change, if you have more that reason to change, which means more than one actor requiring changes.
- It's about actors (user + role) that will require changes on that classes.
- The actor is the single source of change of that responsibility.



SRP is *fundamentally* about separating levels of abstraction

Open / Closed

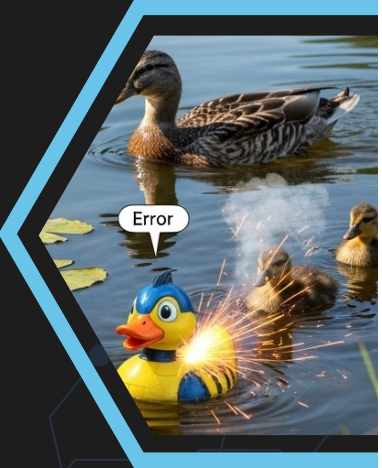
- Change behavior of the class (Open) without changing the source code of the class (Closed).
- The *trick* is to abstract and invert dependencies.
- It only protects you from changes that you know can happen. We have 2 solutions:
 - **BDUP**: Big Design Up-front. Proactive
 - **Agile design**. Iterations, evolutionary design, TDD, etc. Reactive
- Is not possible nor practical to make the whole system follow the OCP.



OCP is about protecting a stable, high-level abstraction from the volatile of its extensions.

Liskov substitution

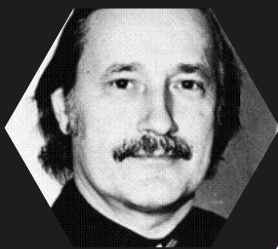
- Substitution: methods in derivative classes should not **shock** the clients changing the behavior of the parent.
- Sub-typing is different in Dynamic and Static Languages. Inheritance vs Duck typing (🦆)
- Every violation of the LSP is a potential violation of the OCP.
- It's about types. For subtypes, IS-A is not enough, CAN-BE-USED-AS is a better relationship.
- Software representatives are not what they represent! e.g. numbers: Integer IS-A Real number.



LSP enforces the integrity of an abstraction by ensuring all its implementations are consistent.

History

Christopher Strachey



Luca Cardelli



Alan Kay



Barbara Liskov



Dahl & Nygaard



Peter Wegner

1967

1970

1985

1988



Interface Segregation

- Decomposing large, incoherent abstractions into smaller, more cohesive ones.
- Interfaces are more related with its users than with its implementers
- Inheritance is the strongest physical relation but the weakest logical relation.
- Composition, in contrast, creates a stronger logical relationship.



ISP focus on preventing clients from depending on modules they don't need.

Dependency Inversion

- Runtime or flow of control vs source code or compile time dependencies.
- Allows to create boundaries
- Low level modules should depend on high level modules.



DIP means to separate high-level policies from low-level details by mediating them through an abstraction.